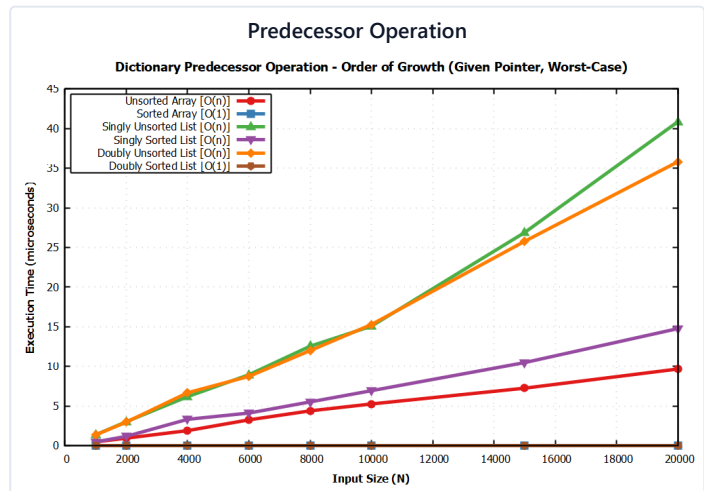
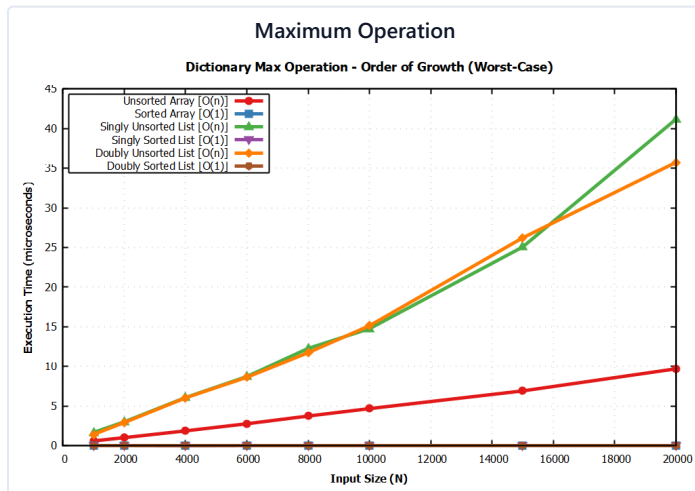
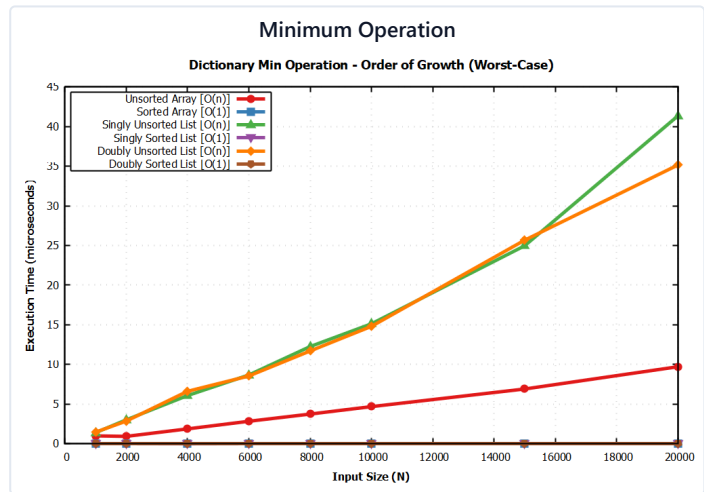
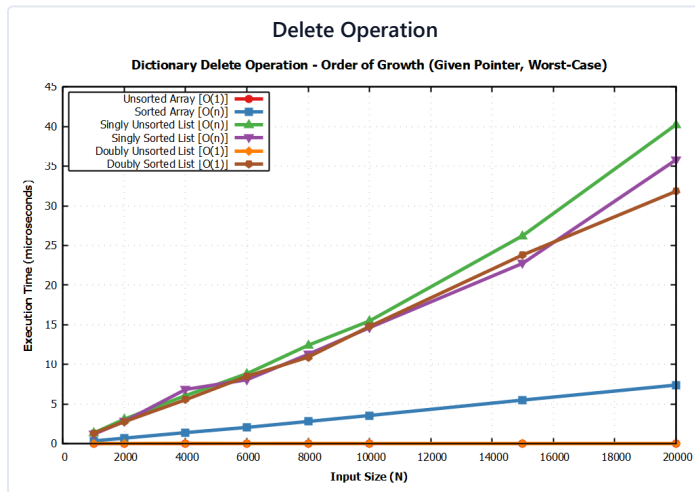
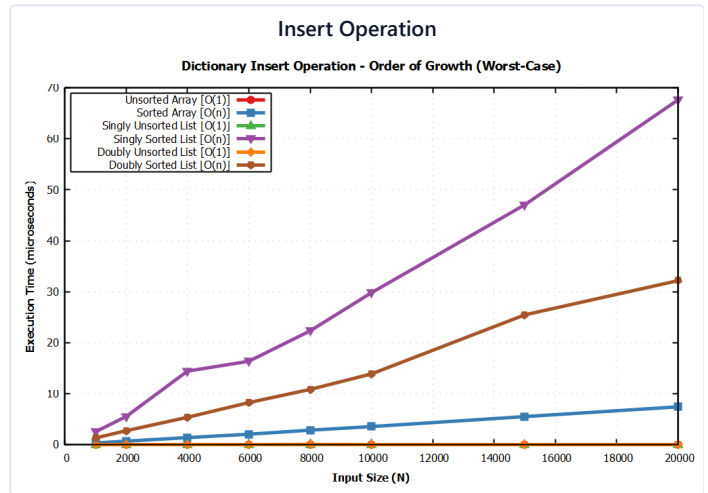
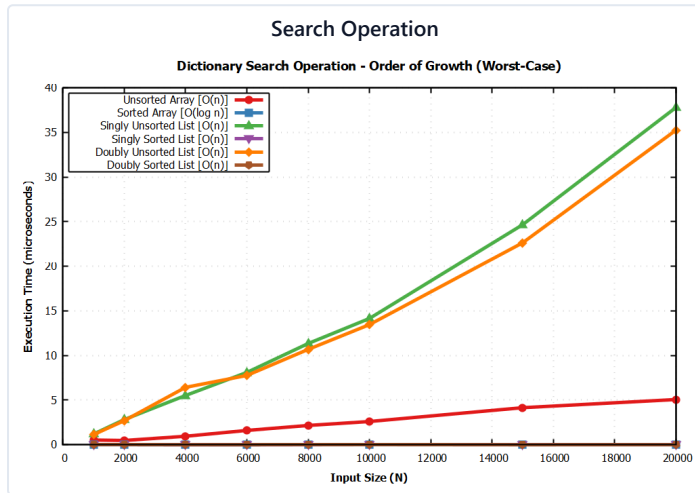


# Dictionary Operations

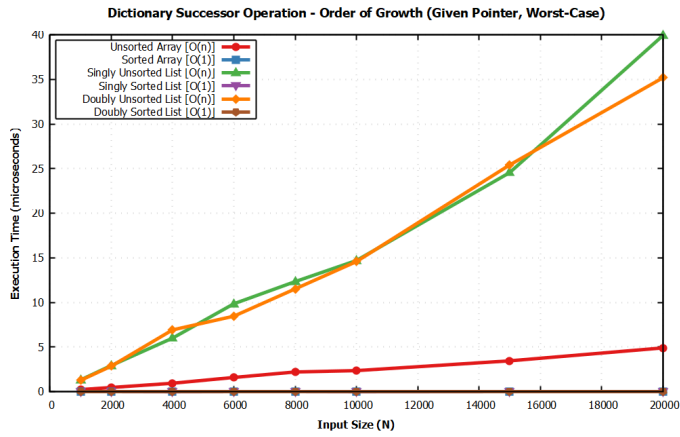
## 1. Asymptotic Worst-Case Complexity Analysis

| Data Structure       | Search      | Insert | Delete | Min    | Max    | Predecessor | Successor |
|----------------------|-------------|--------|--------|--------|--------|-------------|-----------|
| Unsorted Array       | $O(n)$      | $O(1)$ | $O(1)$ | $O(n)$ | $O(n)$ | $O(n)$      | $O(n)$    |
| Sorted Array         | $O(\log n)$ | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ | $O(1)$      | $O(1)$    |
| Singly Unsorted List | $O(n)$      | $O(1)$ | $O(n)$ | $O(n)$ | $O(n)$ | $O(n)$      | $O(n)$    |
| Singly Sorted List   | $O(n)$      | $O(n)$ | $O(n)$ | $O(1)$ | $O(1)$ | $O(n)$      | $O(1)$    |
| Doubly Unsorted List | $O(n)$      | $O(1)$ | $O(1)$ | $O(n)$ | $O(n)$ | $O(n)$      | $O(n)$    |
| Doubly Sorted List   | $O(n)$      | $O(n)$ | $O(1)$ | $O(1)$ | $O(1)$ | $O(1)$      | $O(1)$    |

## 2. Empirical Performance Plots



## Successor Operation



## 3. Source Code

### dictionary.h

```
#ifndef DICTIONARY_H
#define DICTIONARY_H
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <time.h>
// -----
// 1. UNSORTED ARRAY
// -----
typedef struct {
    int *data;
    int capacity;
    int size;
} UnsortedArray;
UnsortedArray* ua_create(int capacity);
void ua_free(UnsortedArray *ua);
int ua_search(UnsortedArray *ua, int key);
void ua_insert(UnsortedArray *ua, int key);
void ua_delete(UnsortedArray *ua, int index);
int ua_min(UnsortedArray *ua);
int ua_max(UnsortedArray *ua);
int ua_predecessor(UnsortedArray *ua, int index);
int ua_successor(UnsortedArray *ua, int index);
// -----
// 2. SORTED ARRAY
// -----
typedef struct {
    int *data;
    int capacity;
    int size;
} SortedArray;
SortedArray* sa_create(int capacity);
void sa_free(SortedArray *sa);
int sa_search(SortedArray *sa, int key);
void sa_insert(SortedArray *sa, int key);
void sa_delete(SortedArray *sa, int index);
int sa_min(SortedArray *sa);
int sa_max(SortedArray *sa);
int sa_predecessor(SortedArray *sa, int index);
int sa_successor(SortedArray *sa, int index);
// -----
// 3. SINGLY LINKED UNSORTED LIST
// -----
typedef struct SNode {
    int key;
    struct SNode *next;
} SNode;
typedef struct {
    SNode *head;
    int size;
} SinglyUnsortedList;
SinglyUnsortedList* su_create(void);
void su_free(SinglyUnsortedList *list);
SNode* su_search(SinglyUnsortedList *list, int key);
SNode* su_insert(SinglyUnsortedList *list, int key);
void su_delete(SinglyUnsortedList *list, SNode *node);
SNode* su_min(SinglyUnsortedList *list);
SNode* su_max(SinglyUnsortedList *list);
SNode* su_predecessor(SinglyUnsortedList *list, SNode *node);
SNode* su_successor(SinglyUnsortedList *list, SNode *node);
// -----
// 4. SINGLY LINKED SORTED LIST
```

```
// -----
typedef struct {
    SNode *head;
    SNode *tail;
    int size;
} SinglySortedList;
SinglySortedList* ss_create(void);
void ss_free(SinglySortedList *list);
SNode* ss_search(SinglySortedList *list, int key);
SNode* ss_insert(SinglySortedList *list, int key);
void ss_delete(SinglySortedList *list, SNode *node);
SNode* ss_min(SinglySortedList *list);
SNode* ss_max(SinglySortedList *list);
SNode* ss_predecessor(SinglySortedList *list, SNode *node);
SNode* ss_successor(SinglySortedList *list, SNode *node);
// -----
// 5. DOUBLY LINKED UNSORTED LIST
// -----
typedef struct DNode {
    int key;
    struct DNode *prev;
    struct DNode *next;
} DNode;
typedef struct {
    DNode *head;
    DNode *tail;
    int size;
} DoublyUnsortedList;
DoublyUnsortedList* du_create(void);
void du_free(DoublyUnsortedList *list);
DNode* du_search(DoublyUnsortedList *list, int key);
DNode* du_insert(DoublyUnsortedList *list, int key);
void du_delete(DoublyUnsortedList *list, DNode *node);
DNode* du_min(DoublyUnsortedList *list);
DNode* du_max(DoublyUnsortedList *list);
DNode* du_predecessor(DoublyUnsortedList *list, DNode *node);
DNode* du_successor(DoublyUnsortedList *list, DNode *node);
// -----
// 6. DOUBLY LINKED SORTED LIST
// -----
typedef struct {
    DNode *head;
    DNode *tail;
    int size;
} DoublySortedList;
DoublySortedList* ds_create(void);
void ds_free(DoublySortedList *list);
DNode* ds_search(DoublySortedList *list, int key);
DNode* ds_insert(DoublySortedList *list, int key);
void ds_delete(DoublySortedList *list, DNode *node);
DNode* ds_min(DoublySortedList *list);
DNode* ds_max(DoublySortedList *list);
DNode* ds_predecessor(DoublySortedList *list, DNode *node);
DNode* ds_successor(DoublySortedList *list, DNode *node);
#endif // DICTIONARY_H
```

### unsorted\_array.c

```
#include "dictionary.h"
UnsortedArray* ua_create(int capacity) {
    UnsortedArray *ua = (UnsortedArray*)malloc(sizeof(UnsortedArray));
    ua->capacity = capacity;
    ua->size = 0;
    ua->data = (int*)malloc(sizeof(int) * capacity);
    return ua;
}
```

```

}
void ua_free(UnsortedArray *ua) {
    if (ua) {
        free(ua->data);
        free(ua);
    }
}
int ua_search(UnsortedArray *ua, int key) {
    for (int i = 0; i < ua->size; i++) {
        if (ua->data[i] == key) return i;
    }
    return -1;
}
void ua_insert(UnsortedArray *ua, int key) {
    if (ua->size >= ua->capacity) {
        ua->capacity *= 2;
        ua->data = (int*)realloc(ua->data, sizeof(int) * ua->capacity);
    }
    ua->data[ua->size++] = key;
}
void ua_delete(UnsortedArray *ua, int index) {
    if (index < 0 || index >= ua->size) return;
    // Fast O(1) swap delete for unsorted array
    ua->data[index] = ua->data[ua->size - 1];
    ua->size--;
}
int ua_min(UnsortedArray *ua) {
    if (ua->size == 0) return -1;
    int min_val = ua->data[0];
    for (int i = 1; i < ua->size; i++) {
        if (ua->data[i] < min_val) min_val = ua->data[i];
    }
    return min_val;
}
int ua_max(UnsortedArray *ua) {
    if (ua->size == 0) return -1;
    int max_val = ua->data[0];
    for (int i = 1; i < ua->size; i++) {
        if (ua->data[i] > max_val) max_val = ua->data[i];
    }
    return max_val;
}
int ua_predecessor(UnsortedArray *ua, int index) {
    if (index < 0 || index >= ua->size) return -1;
    int key = ua->data[index];
    int pred_val = -1;
    bool found = false;
    for (int i = 0; i < ua->size; i++) {
        if (ua->data[i] < key) {
            if (!found || ua->data[i] > pred_val) {
                pred_val = ua->data[i];
                found = true;
            }
        }
    }
    return found ? pred_val : -1;
}
int ua_successor(UnsortedArray *ua, int index) {
    if (index < 0 || index >= ua->size) return -1;
    int key = ua->data[index];
    int succ_val = -1;
    bool found = false;
    for (int i = 0; i < ua->size; i++) {
        if (ua->data[i] > key) {
            if (!found || ua->data[i] < succ_val) {
                succ_val = ua->data[i];
                found = true;
            }
        }
    }
    return found ? succ_val : -1;
}
}

```

#### sorted\_array.c

```

#include "dictionary.h"
SortedArray* sa_create(int capacity) {
    SortedArray *sa = (SortedArray*)malloc(sizeof(SortedArray));
    sa->capacity = capacity;
    sa->size = 0;
    sa->data = (int*)malloc(sizeof(int) * capacity);
    return sa;
}
void sa_free(SortedArray *sa) {
    if (sa) {
        free(sa->data);
        free(sa);
    }
}
int sa_search(SortedArray *sa, int key) {
    int low = 0, high = sa->size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (sa->data[mid] == key) return mid;
        else if (sa->data[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
}

```

```

}
return -1;
}
void sa_insert(SortedArray *sa, int key) {
    if (sa->size >= sa->capacity) {
        sa->capacity *= 2;
        sa->data = (int*)realloc(sa->data, sizeof(int) * sa->capacity);
    }
    int pos = 0;
    while (pos < sa->size && sa->data[pos] < key) {
        pos++;
    }
    for (int i = sa->size; i > pos; i--) {
        sa->data[i] = sa->data[i - 1];
    }
    sa->data[pos] = key;
    sa->size++;
}
void sa_delete(SortedArray *sa, int index) {
    if (index < 0 || index >= sa->size) return;
    for (int i = index; i < sa->size - 1; i++) {
        sa->data[i] = sa->data[i + 1];
    }
    sa->size--;
}
int sa_min(SortedArray *sa) {
    if (sa->size == 0) return -1;
    return sa->data[0];
}
int sa_max(SortedArray *sa) {
    if (sa->size == 0) return -1;
    return sa->data[sa->size - 1];
}
int sa_predecessor(SortedArray *sa, int index) {
    if (index <= 0 || index >= sa->size) return -1;
    return sa->data[index - 1];
}
int sa_successor(SortedArray *sa, int index) {
    if (index < 0 || index >= sa->size - 1) return -1;
    return sa->data[index + 1];
}
}

```

#### singly\_unsorted.c

```

#include "dictionary.h"
SinglyUnsortedList* su_create(void) {
    SinglyUnsortedList *list = (SinglyUnsortedList*)malloc(sizeof(SinglyUnsortedList));
    list->head = NULL;
    list->size = 0;
    return list;
}
void su_free(SinglyUnsortedList *list) {
    if (!list) return;
    SNode *curr = list->head;
    while (curr) {
        SNode *next = curr->next;
        free(curr);
        curr = next;
    }
    free(list);
}
SNode* su_search(SinglyUnsortedList *list, int key) {
    SNode *curr = list->head;
    while (curr) {
        if (curr->key == key) return curr;
        curr = curr->next;
    }
    return NULL;
}
SNode* su_insert(SinglyUnsortedList *list, int key) {
    SNode *node = (SNode*)malloc(sizeof(SNode));
    node->key = key;
    node->next = list->head;
    list->head = node;
    list->size++;
    return node;
}
void su_delete(SinglyUnsortedList *list, SNode *node) {
    if (!list || !list->head || !node) return;
    if (list->head == node) {
        list->head = node->next;
        free(node);
        list->size--;
        return;
    }
    SNode *curr = list->head;
    while (curr && curr->next != node) {
        curr = curr->next;
    }
    if (curr) {
        curr->next = node->next;
        free(node);
        list->size--;
    }
}

```

```

SNode* su_min(SinglyUnsortedList *list) {
    if (!list || !list->head) return NULL;
    SNode *min_node = list->head;
    SNode *curr = list->head->next;
    while (curr) {
        if (curr->key < min_node->key) min_node = curr;
        curr = curr->next;
    }
    return min_node;
}
SNode* su_max(SinglyUnsortedList *list) {
    if (!list || !list->head) return NULL;
    SNode *max_node = list->head;
    SNode *curr = list->head->next;
    while (curr) {
        if (curr->key > max_node->key) max_node = curr;
        curr = curr->next;
    }
    return max_node;
}
SNode* su_predecessor(SinglyUnsortedList *list, SNode *node) {
    if (!list || !list->head || !node) return NULL;
    int target_key = node->key;
    SNode *pred = NULL;
    SNode *curr = list->head;
    while (curr) {
        if (curr->key < target_key) {
            if (!pred || curr->key > pred->key) {
                pred = curr;
            }
        }
        curr = curr->next;
    }
    return pred;
}
SNode* su_successor(SinglyUnsortedList *list, SNode *node) {
    if (!list || !list->head || !node) return NULL;
    int target_key = node->key;
    SNode *succ = NULL;
    SNode *curr = list->head;
    while (curr) {
        if (curr->key > target_key) {
            if (!succ || curr->key < succ->key) {
                succ = curr;
            }
        }
        curr = curr->next;
    }
    return succ;
}

```

#### singly\_sorted.c

```

#include "dictionary.h"
SinglySortedList* ss_create(void) {
    SinglySortedList *list = (SinglySortedList*)malloc(sizeof(SinglySortedList));
    list->head = NULL;
    list->tail = NULL;
    list->size = 0;
    return list;
}
void ss_free(SinglySortedList *list) {
    if (!list) return;
    SNode *curr = list->head;
    while (curr) {
        SNode *next = curr->next;
        free(curr);
        curr = next;
    }
    free(list);
}
SNode* ss_search(SinglySortedList *list, int key) {
    SNode *curr = list->head;
    while (curr && curr->key <= key) {
        if (curr->key == key) return curr;
        curr = curr->next;
    }
    return NULL;
}
SNode* ss_insert(SinglySortedList *list, int key) {
    SNode *node = (SNode*)malloc(sizeof(SNode));
    node->key = key;
    node->next = NULL;
    if (!list->head) {
        list->head = node;
        list->tail = node;
    } else if (key < list->head->key) {
        node->next = list->head;
        list->head = node;
    } else {
        SNode *curr = list->head;
        while (curr->next && curr->next->key <= key) {
            curr = curr->next;
        }
        node->next = curr->next;
        curr->next = node;
    }
}

```

```

        if (!node->next) {
            list->tail = node;
        }
    }
    list->size++;
    return node;
}
void ss_delete(SinglySortedList *list, SNode *node) {
    if (!list || !list->head || !node) return;
    if (list->head == node) {
        list->head = node->next;
        if (!list->head) list->tail = NULL;
        free(node);
        list->size--;
        return;
    }
    SNode *curr = list->head;
    while (curr && curr->next != node) {
        curr = curr->next;
    }
    if (curr) {
        curr->next = node->next;
        if (!curr->next) list->tail = curr;
        free(node);
        list->size--;
    }
}
SNode* ss_min(SinglySortedList *list) {
    if (!list) return NULL;
    return list->head;
}
SNode* ss_max(SinglySortedList *list) {
    if (!list) return NULL;
    return list->tail;
}
SNode* ss_predecessor(SinglySortedList *list, SNode *node) {
    if (!list || !list->head || !node || list->head == node) return NULL;
    SNode *curr = list->head;
    while (curr && curr->next != node) {
        curr = curr->next;
    }
    return curr;
}
SNode* ss_successor(SinglySortedList *list, SNode *node) {
    if (!node) return NULL;
    return node->next;
}

```

#### doubly\_unsorted.c

```

#include "dictionary.h"
DoublyUnsortedList* du_create(void) {
    DoublyUnsortedList *list = (DoublyUnsortedList*)malloc(sizeof(DoublyUnsortedList));
    list->head = NULL;
    list->tail = NULL;
    list->size = 0;
    return list;
}
void du_free(DoublyUnsortedList *list) {
    if (!list) return;
    DNode *curr = list->head;
    while (curr) {
        DNode *next = curr->next;
        free(curr);
        curr = next;
    }
    free(list);
}
DNode* du_search(DoublyUnsortedList *list, int key) {
    DNode *curr = list->head;
    while (curr) {
        if (curr->key == key) return curr;
        curr = curr->next;
    }
    return NULL;
}
DNode* du_insert(DoublyUnsortedList *list, int key) {
    DNode *node = (DNode*)malloc(sizeof(DNode));
    node->key = key;
    node->prev = NULL;
    node->next = list->head;
    if (list->head) {
        list->head->prev = node;
    } else {
        list->tail = node;
    }
    list->head = node;
    list->size++;
    return node;
}
void du_delete(DoublyUnsortedList *list, DNode *node) {
    if (!list || !node) return;
    if (node->prev) {
        node->prev->next = node->next;
    } else {

```

```

        list->head = node->next;
    }
    if (node->next) {
        node->next->prev = node->prev;
    } else {
        list->tail = node->prev;
    }
    free(node);
    list->size--;
}
DNode* du_min(DoublyUnsortedList *list) {
    if (!list || !list->head) return NULL;
    DNode *min_node = list->head;
    DNode *curr = list->head->next;
    while (curr) {
        if (curr->key < min_node->key) min_node = curr;
        curr = curr->next;
    }
    return min_node;
}
DNode* du_max(DoublyUnsortedList *list) {
    if (!list || !list->head) return NULL;
    DNode *max_node = list->head;
    DNode *curr = list->head->next;
    while (curr) {
        if (curr->key > max_node->key) max_node = curr;
        curr = curr->next;
    }
    return max_node;
}
DNode* du_predecessor(DoublyUnsortedList *list, DNode *node) {
    if (!list || !list->head || !node) return NULL;
    int target_key = node->key;
    DNode *pred = NULL;
    DNode *curr = list->head;
    while (curr) {
        if (curr->key < target_key) {
            if (!pred || curr->key > pred->key) {
                pred = curr;
            }
        }
        curr = curr->next;
    }
    return pred;
}
DNode* du_successor(DoublyUnsortedList *list, DNode *node) {
    if (!list || !list->head || !node) return NULL;
    int target_key = node->key;
    DNode *succ = NULL;
    DNode *curr = list->head;
    while (curr) {
        if (curr->key > target_key) {
            if (!succ || curr->key < succ->key) {
                succ = curr;
            }
        }
        curr = curr->next;
    }
    return succ;
}

```

### doubly\_sorted.c

```

#include "dictionary.h"
DoublySortedList* ds_create(void) {
    DoublySortedList *list = (DoublySortedList*)malloc(sizeof(DoublySortedList));
    list->head = NULL;
    list->tail = NULL;
    list->size = 0;
    return list;
}
void ds_free(DoublySortedList *list) {
    if (!list) return;
    DNode *curr = list->head;
    while (curr) {
        DNode *next = curr->next;
        free(curr);
        curr = next;
    }
    free(list);
}
DNode* ds_search(DoublySortedList *list, int key) {
    DNode *curr = list->head;
    while (curr && curr->key <= key) {
        if (curr->key == key) return curr;
        curr = curr->next;
    }
    return NULL;
}
DNode* ds_insert(DoublySortedList *list, int key) {
    DNode *node = (DNode*)malloc(sizeof(DNode));
    node->key = key;
    node->prev = NULL;
    node->next = NULL;
    if (!list->head) {
        list->head = node;
    }
}

```

```

        list->tail = node;
    } else if (key < list->head->key) {
        node->next = list->head;
        list->head->prev = node;
        list->head = node;
    } else {
        DNode *curr = list->head;
        while (curr->next && curr->next->key <= key) {
            curr = curr->next;
        }
        node->next = curr->next;
        node->prev = curr;
        if (curr->next) {
            curr->next->prev = node;
        } else {
            list->tail = node;
        }
        curr->next = node;
    }
    list->size++;
    return node;
}
void ds_delete(DoublySortedList *list, DNode *node) {
    if (!list || !node) return;
    if (node->prev) {
        node->prev->next = node->next;
    } else {
        list->head = node->next;
    }
    if (node->next) {
        node->next->prev = node->prev;
    } else {
        list->tail = node->prev;
    }
    free(node);
    list->size--;
}
DNode* ds_min(DoublySortedList *list) {
    if (!list) return NULL;
    return list->head;
}
DNode* ds_max(DoublySortedList *list) {
    if (!list) return NULL;
    return list->tail;
}
DNode* ds_predecessor(DoublySortedList *list, DNode *node) {
    if (!node) return NULL;
    return node->prev;
}
DNode* ds_successor(DoublySortedList *list, DNode *node) {
    if (!node) return NULL;
    return node->next;
}

```

### main.c

```

#include "dictionary.h"
#ifdef _WIN32 || defined(_WIN64)
#include <windows.h>
static double get_time_us(void) {
    LARGE_INTEGER freq, count;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&count);
    return (double)count.QuadPart * 1000000.0 / (double)freq.QuadPart;
}
#else
#include <time.h>
static double get_time_us(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec * 1000000.0 + (double)ts.tv_nsec / 1000.0;
}
#endif
// Array of N values to test
static const int SIZES[] = {1000, 2000, 4000, 6000, 8000, 10000, 15000, 20000};
static const int NUM_SIZES = sizeof(SIZES) / sizeof(SIZES[0]);
// Ensure directory exists helper
void make_data_dir(void) {
#ifdef _WIN32 || defined(_WIN64)
    system("if not exist data mkdir data");
    system("if not exist plots mkdir plots");
#else
    system("mkdir -p data plots");
#endif
}
int main(void) {
    make_data_dir();
    printf("Starting Dictionary Operations Benchmarks...\n");
    FILE *f_search = fopen("data/search.csv", "w");
    FILE *f_insert = fopen("data/insert.csv", "w");
    FILE *f_delete = fopen("data/delete.csv", "w");
    FILE *f_min = fopen("data/min.csv", "w");
    FILE *f_max = fopen("data/max.csv", "w");
    FILE *f_pred = fopen("data/predecessor.csv", "w");
}

```

```

FILE *f_succ = fopen("data/successor.csv", "w");
const char *csv_header = "N,UnsortedArray,SortedArray,SinglyUnsorted,SinglySorted,DoublyUnsorted,DoublySorted\n";
fputs(csv_header, f_search);
fputs(csv_header, f_insert);
fputs(csv_header, f_delete);
fputs(csv_header, f_min);
fputs(csv_header, f_max);
fputs(csv_header, f_pred);
fputs(csv_header, f_succ);
for (int s = 0; s < NUM_SIZES; s++) {
    int N = SIZES[s];
    printf("Benchmarking size N = %d...\n", N);
    double t_ua[7] = {0}, t_sa[7] = {0};
    double t_su[7] = {0}, t_ss[7] = {0};
    double t_du[7] = {0}, t_ds[7] = {0};
    int REPS_01 = 10000;
    int REPS_ON = 50;
    // -----
    // 1. Unsorted Array
    // -----
    {
        UnsortedArray *ua = ua_create(N + 100);
        for (int i = 0; i < N; i++) ua_insert(ua, i * 2);
        // Search (Worst case: key not found)
        double start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) ua_search(ua, -1);
        t_ua[0] = (get_time_us() - start) / REPS_ON;
        // Insert (Worst case: append)
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) {
            ua_insert(ua, 999999);
            ua->size--; // revert
        }
        t_ua[1] = (get_time_us() - start) / REPS_01;
        // Delete (Swap with last element O(1))
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) {
            int last_val = ua->data[ua->size - 1];
            ua_delete(ua, N / 2);
            ua_insert(ua, last_val); // restore
        }
        t_ua[2] = (get_time_us() - start) / REPS_01;
        // Min / Max
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) ua_min(ua);
        t_ua[3] = (get_time_us() - start) / REPS_ON;
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) ua_max(ua);
        t_ua[4] = (get_time_us() - start) / REPS_ON;
        // Predecessor / Successor
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) ua_predecessor(ua, N / 2);
        t_ua[5] = (get_time_us() - start) / REPS_ON;
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) ua_successor(ua, N / 2);
        t_ua[6] = (get_time_us() - start) / REPS_ON;
        ua_free(ua);
    }
    // -----
    // 2. Sorted Array
    // -----
    {
        SortedArray *sa = sa_create(N + 100);
        for (int i = 0; i < N; i++) sa_insert(sa, i * 2);
        // Search (Binary search)
        double start = get_time_us();
        for (int r = 0; r < REPS_01; r++) sa_search(sa, -1);
        t_sa[0] = (get_time_us() - start) / REPS_01;
        // Insert (Worst case: shift all elements)
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) {
            sa_insert(sa, -1);
            sa_delete(sa, 0); // revert
        }
        t_sa[1] = (get_time_us() - start) / REPS_ON;
        // Delete (Worst case: shift all elements)
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) {
            sa_delete(sa, 0);
            sa_insert(sa, 0); // restore
        }
        t_sa[2] = (get_time_us() - start) / REPS_ON;
        // Min / Max
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) sa_min(sa);
        t_sa[3] = (get_time_us() - start) / REPS_01;
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) sa_max(sa);
        t_sa[4] = (get_time_us() - start) / REPS_01;
        // Predecessor / Successor

```

```

start = get_time_us();
for (int r = 0; r < REPS_01; r++) sa_predecessor(sa, N / 2);
t_sa[5] = (get_time_us() - start) / REPS_01;
start = get_time_us();
for (int r = 0; r < REPS_01; r++) sa_successor(sa, N / 2);
t_sa[6] = (get_time_us() - start) / REPS_01;
sa_free(sa);
}
// -----
// 3. Singly Linked Unsorted List
// -----
{
    SinglyUnsortedList *su = su_create();
    for (int i = 0; i < N; i++) su_insert(su, i * 2);
    // Search
    double start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) su_search(su, -1);
    t_su[0] = (get_time_us() - start) / REPS_ON;
    // Insert (At head)
    start = get_time_us();
    for (int r = 0; r < REPS_01; r++) {
        SNode *n = su_insert(su, -1);
        su_delete(su, n);
    }
    t_su[1] = (get_time_us() - start) / REPS_01;
    // Delete (Delete target at tail - worst case)
    SNode *tail_node = su_search(su, 0); // node 0 is at tail because inserted ascending
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) {
        // measure search for predecessor + delete simulation
        su_predecessor(su, tail_node);
    }
    t_su[2] = (get_time_us() - start) / REPS_ON;
    // Min / Max
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) su_min(su);
    t_su[3] = (get_time_us() - start) / REPS_ON;
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) su_max(su);
    t_su[4] = (get_time_us() - start) / REPS_ON;
    // Predecessor / Successor
    SNode *mid_node = su_search(su, N);
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) su_predecessor(su, mid_node);
    t_su[5] = (get_time_us() - start) / REPS_ON;
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) su_successor(su, mid_node);
    t_su[6] = (get_time_us() - start) / REPS_ON;
    su_free(su);
}
// -----
// 4. Singly Linked Sorted List
// -----
{
    SinglySortedList *ss = ss_create();
    for (int i = 0; i < N; i++) ss_insert(ss, i * 2);
    // Search
    double start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) ss_search(ss, -1);
    t_ss[0] = (get_time_us() - start) / REPS_ON;
    // Insert (at tail)
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) {
        SNode *n = ss_insert(ss, 2 * N + 2);
        ss_delete(ss, n);
    }
    t_ss[1] = (get_time_us() - start) / REPS_ON;
    // Delete (given pointer to tail node)
    SNode *tail_node = ss->tail;
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) {
        ss_predecessor(ss, tail_node); // finding predecessor or dominates delete
    }
    t_ss[2] = (get_time_us() - start) / REPS_ON;
    // Min / Max
    start = get_time_us();
    for (int r = 0; r < REPS_01; r++) ss_min(ss);
    t_ss[3] = (get_time_us() - start) / REPS_01;
    start = get_time_us();
    for (int r = 0; r < REPS_01; r++) ss_max(ss);
    t_ss[4] = (get_time_us() - start) / REPS_01;
    // Predecessor / Successor
    SNode *mid_node = ss_search(ss, N);
    start = get_time_us();
    for (int r = 0; r < REPS_ON; r++) ss_predecessor(ss, mid_node);

```

```

        t_ss[5] = (get_time_us() - start) / REPS_ON;
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) ss_successor(ss, mid_
node);

        t_ss[6] = (get_time_us() - start) / REPS_01;
        ss_free(ss);
    }
    // -----
    // 5. Doubly Linked Unsorted List
    // -----
    {
        DoublyUnsortedList *du = du_create();
        for (int i = 0; i < N; i++) du_insert(du, i * 2);
        // Search
        double start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) du_search(du, -1);
        t_du[0] = (get_time_us() - start) / REPS_ON;
        // Insert (At head)
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) {
            DNode *n = du_insert(du, -1);
            du_delete(du, n);
        }
        t_du[1] = (get_time_us() - start) / REPS_01;
        // Delete (Given pointer, 0(1))
        DNode *mid_node = du_search(du, N);
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) {
            DNode *n = du_insert(du, 999999);
            du_delete(du, n);
        }
        t_du[2] = (get_time_us() - start) / REPS_01;
        // Min / Max
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) du_min(du);
        t_du[3] = (get_time_us() - start) / REPS_ON;
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) du_max(du);
        t_du[4] = (get_time_us() - start) / REPS_ON;
        // Predecessor / Successor
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) du_predecessor(du, mi
d_node);

        t_du[5] = (get_time_us() - start) / REPS_ON;
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) du_successor(du, mid_
node);

        t_du[6] = (get_time_us() - start) / REPS_ON;
        du_free(du);
    }
    // -----
    // 6. Doubly Linked Sorted List
    // -----
    {
        DoublySortedList *ds = ds_create();
        for (int i = 0; i < N; i++) ds_insert(ds, i * 2);
        // Search
        double start = get_time_us();

```

```

        for (int r = 0; r < REPS_ON; r++) ds_search(ds, -1);
        t_ds[0] = (get_time_us() - start) / REPS_ON;
        // Insert (at tail)
        start = get_time_us();
        for (int r = 0; r < REPS_ON; r++) {
            DNode *n = ds_insert(ds, 2 * N + 2);
            ds_delete(ds, n);
        }
        t_ds[1] = (get_time_us() - start) / REPS_ON;
        // Delete (0(1))
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) {
            DNode *n = ds_insert(ds, 2 * N + 2);
            ds_delete(ds, n);
        }
        t_ds[2] = (get_time_us() - start) / REPS_01;
        // Min / Max
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) ds_min(ds);
        t_ds[3] = (get_time_us() - start) / REPS_01;
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) ds_max(ds);
        t_ds[4] = (get_time_us() - start) / REPS_01;
        // Predecessor / Successor
        DNode *mid_node = ds_search(ds, N);
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) ds_predecessor(ds, mi
d_node);

        t_ds[5] = (get_time_us() - start) / REPS_01;
        start = get_time_us();
        for (int r = 0; r < REPS_01; r++) ds_successor(ds, mid_
node);

        t_ds[6] = (get_time_us() - start) / REPS_01;
        ds_free(ds);
    }
    // Output row to each CSV
    fprintf(f_search, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[0], t_sa[0], t_su[0], t_ss[0], t_du[0], t_ds[0]);
    fprintf(f_insert, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[1], t_sa[1], t_su[1], t_ss[1], t_du[1], t_ds[1]);
    fprintf(f_delete, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[2], t_sa[2], t_su[2], t_ss[2], t_du[2], t_ds[2]);
    fprintf(f_min, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[3], t_sa[3], t_su[3], t_ss[3], t_du[3], t_ds[3]);
    fprintf(f_max, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[4], t_sa[4], t_su[4], t_ss[4], t_du[4], t_ds[4]);
    fprintf(f_pred, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[5], t_sa[5], t_su[5], t_ss[5], t_du[5], t_ds[5]);
    fprintf(f_succ, "%d,%.4f,%.4f,%.4f,%.4f,%.4f,%.4f\n", N,
t_ua[6], t_sa[6], t_su[6], t_ss[6], t_du[6], t_ds[6]);
    }
    fclose(f_search);
    fclose(f_insert);
    fclose(f_delete);
    fclose(f_min);
    fclose(f_max);
    fclose(f_pred);
    fclose(f_succ);
    printf("Benchmarking completed successfully! CSV files exported
to data/\n");
    return 0;
}

```